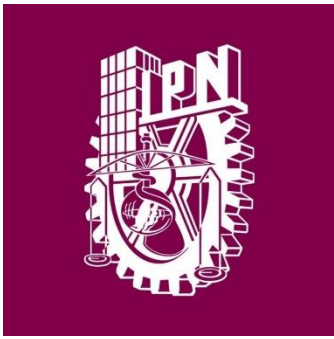




Instituto Politécnico
Nacional
Escuela Superior de
Computo



Profesor: Cortes Galicia Jorge

Materia: Sistemas Operativos

Practica 6

Integrantes:

Vázquez Blancas Cesar Said

Diaz Torres Jonathan Samuel

Aparicio Ponce Roberto Manuel

Corona Hernández Martin Rafael

Introducción Teórica

En los sistemas operativos modernos, como Windows y Linux, los programas se ejecutan como **procesos**. Un proceso es una instancia de un programa en ejecución que incluye los recursos necesarios para su funcionamiento, como su propio espacio de memoria y variables independientes. Los procesos son la base de la multitarea en los sistemas operativos, permitiendo que múltiples programas se ejecuten simultáneamente.

Concepto de Hilos (Threads)

Dentro de cada proceso, existen **hilos** (o *threads*), que son unidades más pequeñas de ejecución. Los hilos comparten el mismo espacio de memoria y recursos del proceso al que pertenecen, lo que permite que diferentes partes de un programa se ejecuten concurrentemente. A estos hilos también se les conoce como "procesos ligeros" debido a que, a diferencia de los procesos independientes, comparten los mismos recursos y son menos costosos para el sistema en términos de uso de memoria y tiempo de CPU.

Los hilos permiten dividir las tareas de un programa en unidades menores, lo que mejora la eficiencia de los sistemas modernos, especialmente aquellos que poseen múltiples núcleos de CPU. Por ejemplo, un navegador web puede utilizar varios hilos: uno para la interfaz de usuario, otro para el procesamiento de imágenes, y otros para manejar múltiples pestañas. Así, si un hilo se bloquea debido a una operación compleja, el resto del programa puede seguir ejecutándose.

Diferencias entre Procesos e Hilos

- **Procesos:** Son independientes unos de otros, cada uno tiene su propio espacio de memoria y requieren una mayor cantidad de recursos del sistema. En un sistema operativo, la creación de un proceso implica una sobrecarga mayor que la creación de un hilo.
- **Hilos:** Los hilos de un mismo proceso comparten memoria y recursos, lo que permite que su creación y administración sea más rápida y eficiente en cuanto a recursos. Sin embargo, debido a que comparten memoria, los hilos pueden entrar en conflicto entre sí si no se gestionan correctamente, lo que puede causar problemas de *sincronización* y *condiciones de carrera*.

Administración de Hilos en Windows y Linux

Tanto en Windows como en Linux, los sistemas operativos proporcionan herramientas y funciones para la administración y monitoreo de hilos:

- **Windows:** En el sistema operativo Windows, los hilos se gestionan a través del **Administrador de Tareas** y herramientas avanzadas como **Process Explorer**. En el Administrador de Tareas, en la pestaña de "Detalles", se

puede observar cuántos hilos están activos en cada proceso. Process Explorer ofrece un análisis más detallado de los hilos, permitiendo ver los recursos específicos que cada hilo consume y, en algunos casos, permite gestionar hilos de manera independiente.

- **Linux:** En Linux, los hilos pueden observarse mediante comandos como `top` y `htop`. Ambos permiten monitorear el sistema en tiempo real, mostrando el uso de CPU, memoria y otros recursos tanto para procesos completos como para hilos individuales (al activar la opción de mostrar hilos). También existen herramientas avanzadas como `ps` y `strace`, que permiten un análisis detallado del comportamiento de cada hilo dentro del sistema.

Importancia de los Hilos en la Eficiencia del Sistema

La utilización de hilos permite una ejecución más eficiente de aplicaciones y servicios en sistemas con múltiples núcleos, ya que estos pueden distribuir la carga de trabajo entre sus núcleos, mejorando el rendimiento y la capacidad de respuesta del sistema. Además, los hilos son fundamentales en aplicaciones que requieren realizar múltiples tareas simultáneamente, como servidores web y aplicaciones gráficas.

Sin embargo, el manejo de hilos también introduce desafíos, como la **sincronización** (para evitar que los hilos interfieran entre sí) y la **gestión de estados compartidos**. Si los hilos no se gestionan adecuadamente, pueden provocar problemas como *bloqueos mutuos* (deadlocks) y *condiciones de carrera* (race conditions), donde dos o más hilos intentan acceder a los mismos recursos simultáneamente y causan resultados inesperados.

En esta práctica, exploraremos cómo el sistema operativo Windows y Linux manejan los hilos, cómo monitorear el rendimiento de los hilos, y cómo entender el impacto que tienen en la ejecución eficiente de los programas.

Sección Linux:

1.- A través de la ayuda en línea que proporciona Linux, investigue el funcionamiento de las funciones: `pthread_create()`, `pthread_join()`, `pthread_self()`, `pthread_exit()`, `scandir()`, `stat()`. Explique los argumentos y retorno de cada función.

1. `pthread_create()`

Esta función de la biblioteca de POSIX es utilizada para crear un nuevo hilo de ejecución.

- **Prototipo:**

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void  
*(*start_routine) (void *), void *arg);
```

- **Argumentos:**

- pthread_t *thread: Apuntador a una variable de tipo pthread_t donde se almacenará el identificador del hilo creado.
- const pthread_attr_t *attr: Especifica los atributos del hilo. Si se pasa NULL, se utilizarán los atributos por defecto.
- void *(*start_routine) (void *): Apuntador a la función que ejecutará el hilo. Esta función debe aceptar y devolver un void *.
- void *arg: Argumento que será pasado a la función start_routine.

- **Retorno:** Retorna 0 si el hilo se crea correctamente. En caso de error, retorna un código de error distinto de cero.

2. pthread_join()

Esta función permite esperar a que un hilo específico termine su ejecución, lo que permite sincronizar la finalización de un hilo con otro.

- **Prototipo:**

```
int pthread_join(pthread_t thread, void **retval);
```

- **Argumentos:**

- pthread_t thread: Identificador del hilo que se quiere esperar.
- void **retval: Apuntador para almacenar el valor de retorno de la función de inicio del hilo. Si no se desea recibir el valor, se puede pasar NULL.

- **Retorno:** Retorna 0 si tiene éxito; en caso de error, retorna un código de error.

3. pthread_self()

Esta función devuelve el identificador del hilo que la llama.

- **Prototipo:**

```
pthread_t pthread_self(void);
```

- **Argumentos:** No tiene argumentos.
- **Retorno:** Retorna un valor de tipo pthread_t, que es el identificador del hilo que realiza la llamada.

4. pthread_exit()

Termina la ejecución de un hilo y opcionalmente devuelve un valor de salida.

- **Prototipo:**

```
void pthread_exit(void *retval);
```

- **Argumentos:**

- void *retval: Valor que se retorna como resultado del hilo. Este valor puede ser recuperado por otro hilo que utilice pthread_join para esperar la terminación del hilo actual.

- **Retorno:** No retorna nada, ya que termina la ejecución del hilo que la llama.

5. scandir()

Esta función se utiliza para listar los archivos y directorios dentro de un directorio específico.

- **Prototipo:**

```
int scandir(const char *dir, struct dirent ***namelist, int (*filter)(const struct dirent *),  
int (*compar)(const struct dirent **, const struct dirent **));
```

- **Argumentos:**

- const char *dir: Ruta del directorio que se quiere escanear.
- struct dirent ***namelist: Doble apuntador a una lista de apuntadores a estructuras dirent que contendrá la información de cada archivo o directorio encontrado.
- int (*filter)(const struct dirent *): Función opcional para filtrar los resultados. Si se pasa NULL, se incluirán todos los elementos.
- int (*compar)(const struct dirent **, const struct dirent **): Función opcional para ordenar los resultados. Si se pasa NULL, los elementos no se ordenarán.

- **Retorno:** Retorna el número de elementos encontrados en el directorio o -1 en caso de error.

6. stat()

Esta función se utiliza para obtener información detallada sobre un archivo o directorio, como tamaño, permisos y fechas de modificación.

- **Prototipo:**

```
int stat(const char *pathname, struct stat *buf);
```

- **Argumentos:**
 - `const char *pathname`: Ruta del archivo o directorio del que se quiere obtener información.
 - `struct stat *buf`: Apuntador a una estructura `stat` donde se almacenará la información del archivo.
- **Retorno:** Retorna 0 si tiene éxito; en caso de error, retorna -1.

2. Capture, compile y ejecute el programa de creación de un nuevo hilo en Linux. Observe su funcionamiento.

```
#include <stdio.h>
#include <pthread.h>
void *hilo(void *arg);
int main(void)
{
    pthread_t id_hilo;
    pthread_create(&id_hilo, NULL, (void*)hilo, NULL);
    pthread_join(id_hilo, NULL);
    return 0;
}
void *hilo(void *arg)
{
    printf("Hola mundo desde un hilo en Linux\n");
    return NULL;
}
```

```
said@said-VirtualBox:~/Descargas$ gcc p1.c -o p1
said@said-VirtualBox:~/Descargas$ ./p1
Hola mundo desde un hilo en Linux
```

3. Capture, compile y ejecute el siguiente programa de creación de hilos en Linux. Observe su funcionamiento.

```
#include <stdio.h>
#include <pthread.h>
void *hilo(void *arg);
int main(void)
{
    pthread_t id_hilo;
    char* mensaje="Hola a todos desde el hilo";
    int devolucion_hilo;
    pthread_create(&id_hilo,NULL,hilo,(void*)mensaje);
    pthread_join(id_hilo,(void*)&devolucion_hilo);
    printf("valor = %i\n",devolucion_hilo);
    return 0;
}
void *hilo(void *arg)
{
    char* men;
    int resultado_hilo=0;
    men=(char*)arg;
    printf("%s\n",men);
    resultado_hilo=100;
    pthread_exit((void*)resultado_hilo);
}
```

```
said@said-VirtualBox:~/Descargas$ ./p2
Hola a todos desde el hilo
valor = 100
```

4. A través del sitio MSDN, investigue el funcionamiento de la llamada al sistema CreateThread().

La función CreateThread en Windows es una llamada del sistema que permite crear un nuevo hilo de ejecución dentro de un proceso. Cada hilo se ejecuta de manera independiente y permite que diferentes tareas se ejecuten de forma simultánea en el mismo espacio de memoria.

Descripción de CreateThread

- **Prototipo:**

```
HANDLE CreateThread(  
    LPSECURITY_ATTRIBUTES lpThreadAttributes,  
    SIZE_T dwStackSize,  
    LPTHREAD_START_ROUTINE lpStartAddress,  
    LPVOID lpParameter,  
    DWORD dwCreationFlags,  
    LPDWORD lpThreadId  
);
```

Parámetros

1. **lpThreadAttributes:**

- Es un puntero a una estructura SECURITY_ATTRIBUTES que determina la herencia del identificador del hilo.
- Si es NULL, el hilo no puede ser heredado por otros procesos.
- También establece un descriptor de seguridad para el hilo si se proporciona.

2. **dwStackSize:**

- Tamaño inicial de la pila para el nuevo hilo, en bytes.
- Si se establece en 0, el hilo hereda el tamaño de pila predeterminado del proceso principal.
- Puede ajustarse para optimizar el uso de memoria según las necesidades de la aplicación.

3. **lpStartAddress:**

- Apuntador a la función que ejecutará el hilo, conocida como ThreadProc. Esta función debe tener la firma DWORD WINAPI ThreadProc(LPVOID lpParameter).
- La ejecución del hilo comenzará en esta dirección de función.

4. **lpParameter:**

- Un valor que se pasa como parámetro a la función de inicio del hilo (ThreadProc).

- Se usa para enviar información al hilo nuevo, típicamente un apuntador a una estructura de datos.

5. **dwCreationFlags:**

- Especifica el comportamiento del hilo al crearse.
- Los valores comunes incluyen:
 - 0: El hilo inicia inmediatamente.
 - **CREATE_SUSPENDED:** El hilo se crea en estado suspendido y no comienza a ejecutarse hasta que se llama a **ResumeThread**.

6. **lpThreadId:**

- Es un puntero a una variable que recibe el identificador del hilo.
- Puede usarse para obtener y gestionar el hilo creado en otras partes del programa.

Valor de Retorno

- **Retorno Exitoso:** Retorna un **HANDLE** que representa el hilo creado. Este identificador permite que otras funciones de la API de Windows gestionen o interactúen con el hilo.
- **Error:** Retorna **NULL**. Para obtener más detalles sobre el error, se puede llamar a **GetLastError**.

Uso Típico

La función **CreateThread** se usa comúnmente para crear hilos en aplicaciones de Windows que requieren multitarea o para dividir el trabajo en paralelo. Sin embargo, en Windows moderno, se recomienda utilizar **CreateThread** únicamente en aplicaciones de bajo nivel o para fines específicos del sistema, ya que la API **BeginThread** y el entorno de Windows ofrecen alternativas con una mayor capacidad de gestión de recursos y seguridad.

Observaciones Importantes

- **Gestión del identificador de hilo:** Después de que el hilo termina su ejecución, el identificador del hilo debe cerrarse usando **CloseHandle** para liberar los recursos del sistema.
- **Alternativa recomendada:** Para aplicaciones de alto nivel en C/C++, Microsoft sugiere usar funciones como **_beginthread** y **_beginthreadex** para mejor manejo del almacenamiento local del hilo y optimización de recursos.

5. Capture, compile y ejecute el siguiente programa de creación de hilos en Windows. Observe su funcionamiento.

```
#include <windows.h>
#include <stdio.h>
DWORD WINAPI funcionHilo(LPVOID lpParam);
typedef struct Informacion info;
struct Informacion
{
    int val_1;
    int val_2;
};
int main(void)
{
    DWORD idHilo; /* Identificador del Hilo */
    HANDLE manHilo; /* Manejador del Hilo */
    info argumentos;
    argumentos.val_1=10;
    argumentos.val_2=100;
    // Creacion del hilo
    manHilo=CreateThread(NULL, 0, funcionHilo, &argumentos, 0, &idHilo);

    // Espera la finalización del hilo
    WaitForSingleObject(manHilo, INFINITE);
    printf("Valores al salir del Hilo: %i %i\n", argumentos.val_1, argumentos.val_2);

    // Cierre del manejador del hilo creado
    CloseHandle(manHilo);
    return 0;
}
DWORD WINAPI funcionHilo(LPVOID lpParam)
{
    info *datos=(info *)lpParam;
    printf("Valores al entrar al Hilo: %i %i\n", datos->val_1, datos->val_2);
    datos->val_1*=2;
    datos->val_2*=2;
    return 0;
}
```

```
C:\Users\vbcs1>gcc p1w.c -o p1
```

```
C:\Users\vbcs1>p1
```

```
Valores al entrar al Hilo: 10 100
```

```
Valores al salir del Hilo: 20 200
```

6. Programe una aplicación (tanto en Linux como en Windows), que cree un proceso hijo a partir de un proceso padre, el hijo creado a su vez creará 20 hilos. A su vez cada uno de los 20 hilos creará 15 hilos más. A su vez cada uno de los 15 hilos creará 10 hilos más. Cada uno de los hilos creados imprimirá en pantalla “Práctica 6 Hilo Terminal” si se trata de un hilo terminal o los identificadores de los hilos creados si se trata de un proceso o hilo padre.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
void* thread3(void* arg) {
    printf("\t\tPráctica 6 Hilo Terminal\n");
    pthread_exit(NULL);
}
void* thread2(void* arg) {
    pthread_t Threads3[10];
    printf("\tHilo de nivel 2 ID: %ld\n", pthread_self());
    for (int i = 0; i < 10; i++) {
        pthread_create(&Threads3[i], NULL, thread3, NULL);
    }
    for (int i = 0; i < 10; i++) {
        pthread_join(Threads3[i], NULL);
    }

    pthread_exit(NULL);
}
void* thread1(void* arg) {
    pthread_t Threads3[15];
    printf("Hilo de nivel 1 ID: %ld\n", pthread_self());
    for (int i = 0; i < 15; i++) {
        pthread_create(&Threads3[i], NULL, thread2, NULL);
    }
    for (int i = 0; i < 15; i++) {
        pthread_join(Threads3[i], NULL);
    }
    pthread_exit(NULL);
}
```

```

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        pthread_t Threads2[20];
        for (int i = 0; i < 20; i++) {
            pthread_create(&Threads2[i], NULL, thread1, NULL);
        }
        for (int i = 0; i < 20; i++) {
            pthread_join(Threads2[i], NULL);
        }
    } else {
        wait(0);
    }
    return 0;
}

```

Linux

Salida recortada:

```

said@said-VirtualBox:~/Descargas$ ./hilos
Hilo de nivel 1 ID: 130262126036672
Hilo de nivel 1 ID: 130261993916096
Hilo de nivel 1 ID: 130262115550912
Hilo de nivel 1 ID: 130261962458816
Hilo de nivel 1 ID: 130262084093632
    Hilo de nivel 2 ID: 130261826143936
    Hilo de nivel 2 ID: 130261580777152
    Hilo de nivel 2 ID: 130261920515776
Hilo de nivel 1 ID: 130261951973056
    Hilo de nivel 2 ID: 130261679343296
    Hilo de nivel 2 ID: 130261910030016
    Hilo de nivel 2 ID: 130261889058496
    Hilo de nivel 2 ID: 130261559805632
    Hilo de nivel 2 ID: 130261878572736
Hilo de nivel 1 ID: 130261710800576
Hilo de nivel 1 ID: 130261700314816
    Hilo de nivel 2 ID: 130261857601216
    Hilo de nivel 2 ID: 130261805172416
    Hilo de nivel 2 ID: 130261931001536
    Hilo de nivel 2 ID: 130261847115456
Hilo de nivel 1 ID: 130261899544256
Hilo de nivel 1 ID: 130261794686656

```

Hilo de nivel 1 ID: 130261784200896
Hilo de nivel 1 ID: 130261773715136
Hilo de nivel 1 ID: 130261763229376
Hilo de nivel 1 ID: 130261752743616
Hilo de nivel 1 ID: 130261742257856
Hilo de nivel 1 ID: 130261731772096
Hilo de nivel 1 ID: 130261721286336
 Hilo de nivel 2 ID: 130261836629696
 Hilo de nivel 2 ID: 130261570291392
 Hilo de nivel 2 ID: 130261538834112
 Hilo de nivel 2 ID: 130261389936320
 Hilo de nivel 2 ID: 130261368964800
Hilo de nivel 1 ID: 130262094579392
 Hilo de nivel 2 ID: 130261689829056
 Hilo de nivel 2 ID: 130261815658176
 Hilo de nivel 2 ID: 130261591262912
 Hilo de nivel 2 ID: 130261868086976
 Hilo de nivel 2 ID: 130261941487296
 Hilo de nivel 2 ID: 130261379450560
 Hilo de nivel 2 ID: 130261224261312
 Hilo de nivel 2 ID: 130261972944576
 Hilo de nivel 2 ID: 130261337507520
 Hilo de nivel 2 ID: 130262105065152

 Hilo de nivel 2 ID: 130261224261312
 Hilo de nivel 2 ID: 130261972944576
 Hilo de nivel 2 ID: 130261337507520
 Hilo de nivel 2 ID: 130262105065152
 Práctica 6 Hilo Terminal
 Práctica 6 Hilo Terminal
 Práctica 6 Hilo Terminal
 Práctica 6 Hilo Terminal
 Práctica 6 Hilo Terminal
 Práctica 6 Hilo Terminal
 Práctica 6 Hilo Terminal
 Práctica 6 Hilo Terminal
 Hilo de nivel 2 ID: 130261255718592
 Hilo de nivel 2 ID: 130261245232832
 Hilo de nivel 2 ID: 130261234747072
Hilo de nivel 1 ID: 130262073607872
 Hilo de nivel 2 ID: 130261213775552
Hilo de nivel 1 ID: 130261983430336
 Hilo de nivel 2 ID: 130261358479040
 Hilo de nivel 2 ID: 130261035517632
 Hilo de nivel 2 ID: 130261347993280
 Hilo de nivel 2 ID: 130261192804032
 Hilo de nivel 2 ID: 130261150860992
 Hilo de nivel 2 ID: 130261108917952

Práctica 6 Hilo Terminal
Práctica 6 Hilo Terminal
Práctica 6 Hilo Terminal
Hilo de nivel 2 ID: 130260899202752
Práctica 6 Hilo Terminal
Hilo de nivel 2 ID: 130258885936832
Práctica 6 Hilo Terminal
Hilo de nivel 2 ID: 130260114867904
Práctica 6 Hilo Terminal
Hilo de nivel 2 ID: 130260888716992
Práctica 6 Hilo Terminal
Práctica 6 Hilo Terminal
Práctica 6 Hilo Terminal
Práctica 6 Hilo Terminal
Práctica 6 Hilo Terminal
Práctica 6 Hilo Terminal
Hilo de nivel 2 ID: 130260507035328
Práctica 6 Hilo Terminal
Hilo de nivel 2 ID: 130258854479552
Hilo de nivel 2 ID: 130260072924864
Hilo de nivel 2 ID: 130260093896384

Windows:

Salida Recortada:

HilosPadre.c

```
#include <windows.h>
#include <stdio.h>
int main(int argc, char *argv[]) {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    int i;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
    if(argc != 2) {
        printf("Usar: %s Nombre_programa_hijo\n", argv[0]);
        return 1;
    }
    if(!CreateProcess(NULL, argv[1], NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
        printf("Error al crear proceso hijo(%d)\n", GetLastError());
        return 1;
    }
    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    return 0;
}
```

HilosHijo.c

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
DWORD id;
HANDLE hilo;
DWORD WINAPI thread3(LPVOID arg) {
    printf("\t\tPractica 6 Hilo Terminal\n");
    return 0;
}
DWORD WINAPI thread2(LPVOID arg) {
    printf("\t2. ID: %d\n", GetCurrentThreadId());
    for (int i = 0; i < 10; i++) {
        hilo = CreateThread(NULL, 0, thread3, NULL, 0, &id);
    }
    return 0;
}
DWORD WINAPI thread1(LPVOID arg) {
    printf("1. ID: %d\n", GetCurrentThreadId());
    for (int i = 0; i < 15; i++) {
        hilo = CreateThread(NULL, 0, thread2, NULL, 0, &id);
    }
    return 0;
}
int main() {
    for (int i = 0; i < 20; i++) {
        hilo = CreateThread(NULL, 0, thread1, NULL, 0, &id);
    }
    WaitForSingleObject(hilo, INFINITE);
    CloseHandle(hilo);
    return 0;
}
```

```
C:\Users\vbcs1>padre hijo
1. ID: 19880
1. ID: 24056
1. ID: 18476
1. ID: 5420
    2. ID: 25224
1. ID: 17616
1. ID: 8052
1. ID: 30140
1. ID: 13904
1. ID: 17656
1. ID: 21044
1. ID: 17640
1. ID: 17632
1. ID: 17628
    2. ID: 29132
1. ID: 17708
    2. ID: 15488
    2. ID: 4928
1. ID: 17680
    2. ID: 3156
    2. ID: 25700
1. ID: 17660
    2. ID: 17636
1. ID: 13172
    2. ID: 26128
    2. ID: 25164
    2. ID: 29364
    2. ID: 8276
2. ID: 2552
2. ID: 21428
2. ID: 13588
2. ID: 25016
2. ID: 5568
2. ID: 17668
2. ID: 29860
2. ID: 27724
2. ID: 28384
2. ID: 29892
2. ID: 28312
2. ID: 20880
2. ID: 27188
2. ID: 29448
2. ID: 23768
2. ID: 22548
2. ID: 25576
2. ID: 24012
2. ID: 23816
2. ID: 6972
2. ID: 22932
2. ID: 29772
2. ID: 21656
2. ID: 25228
1. ID: 18972
2. ID: 28612
2. ID: 9756
2. ID: 24248
2. ID: 28352
```


2. ID: 29776
2. ID: 17752
2. ID: 26304
2. ID: 21740
2. ID: 27800
2. ID: 29016
2. ID: 30192
2. ID: 22160
2. ID: 26964
2. ID: 23532
2. ID: 7960
2. ID: 29244
2. ID: 27936
2. ID: 27552
2. ID: 972
2. ID: 24724
 Practica 6 Hilo Terminal
2. ID: 21620
2. ID: 20304
2. ID: 28608
2. ID: 22256
2. ID: 9632
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
2. ID: 12532
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal

2. ID: 2052
2. ID: 31332
2. ID: 9420
2. ID: 30860
2. ID: 25880
2. ID: 29924
2. ID: 12588
2. ID: 31168
2. ID: 10700
2. ID: 27572
2. ID: 10684
2. ID: 6428
 Practica 6 Hilo Terminal
2. ID: 15516
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
1. ID: 13576
 Practica 6 Hilo Terminal
 Practica 6 Hilo Terminal
2. ID: 19476

7. Programe la misma aplicación del punto 5 de la práctica 5 en su sección de Linux pero utilizando hilos (tanto en Linux como en Windows) en vez de procesos, obtenga el tiempo de ejecución del programa con hilos. Compare ambos programas en su tiempo de ejecución (el creado en la práctica 5 y el creado en esta práctica), y dé sus observaciones tanto de funcionamiento como de los tiempos de ejecución resultantes.

LINUX

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <pthread.h>
#include <time.h>

#define N 10

int A[N][N];
int B[N][N];
int resultadoSuma[N][N];
int resultadoResta[N][N];
int resultadoMultiplicacion[N][N];
int resultadoT1[N][N];
int resultadoT2[N][N];

typedef struct {
    int matriz[N][N];
    double inversa[N][N];
    double determinante;
} Inversa;

void imprimirMatriz(int A[N][N]){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%d\t", A[i][j]);
        }
        printf("\n");
    }
}

void escribirMatrizInt(const char *nombre_archivo, int matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%d ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

void escribirMatrizDouble(const char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

void leerYMostrarMatriz(const char *titulo, const char *nombre_archivo) {
    FILE *archivo = fopen(nombre_archivo, "r");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        return;
    }
    printf("\n%s:\n", titulo);
    char linea[256];
    while (fgets(linea, sizeof(linea), archivo)) {
        printf("%s", linea);
    }
    fclose(archivo);
}
```

```

void *sumarMatrices(void *arg) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultadoSuma[i][j] = A[i][j] + B[i][j];
        }
    }
    escribirMatrizInt("suma.txt", resultadoSuma);
    return NULL;
}

void *restarMatrices(void *arg) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultadoResta[i][j] = A[i][j] - B[i][j];
        }
    }
    escribirMatrizInt("resta.txt", resultadoResta);
    return NULL;
}

void *multiplicarMatrices(void *arg) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultadoMultiplicacion[i][j] = 0;
            for (int k = 0; k < N; k++) {
                resultadoMultiplicacion[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    escribirMatrizInt("multiplicacion.txt", resultadoMultiplicacion);
    return NULL;
}

void *transponerMatriz(void *arg) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultadoT1[j][i] = A[i][j];
            resultadoT2[j][i] = B[i][j];
        }
    }
    escribirMatrizInt("transpuesta_A.txt", resultadoT1);
    escribirMatrizInt("transpuesta_B.txt", resultadoT2);
    return NULL;
}

```

```

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++) {
        for (int col = 0; col < n; col++) {
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1) {
                    j = 0;
                    i++;
                }
            }
        }
    }
}

int DeterminanteMatriz(int matriz[N][N], int n) {
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];

    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++) {
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = signo * DeterminanteMatriz(temp, n - 1);
        }
    }
}

```

```

}
void *calcularInversa(void *arg) {
    Inversa *datos = (Inversa *)arg;
    double determinante = (double)DeterminanteMatriz(datos->matriz, N);
    datos->determinante = determinante;
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return NULL;
    }
    double adjunta[N][N];
    AdjuntaMatriz(datos->matriz, adjunta, N);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            datos->inversa[i][j] = adjunta[i][j] / determinante;
        }
    }
    return NULL;
}

void *InversaMatriz(void *arg) {
    Inversa datosA = {.matriz = {0}, .inversa = {0}, .determinante = 0};
    Inversa datosB = {.matriz = {0}, .inversa = {0}, .determinante = 0};
    memcpy(datosA.matriz, A, sizeof(A));
    memcpy(datosB.matriz, B, sizeof(B));
    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, calcularInversa, &datosA);
    pthread_create(&thread2, NULL, calcularInversa, &datosB);
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    if (datosA.determinante != 0) {
        escribirMatrizDouble("inversa_A.txt", datosA.inversa);
    }
    if (datosB.determinante != 0) {
        escribirMatrizDouble("inversa_B.txt", datosB.inversa);
    }
    return NULL;
}

```

```

void *leerArchivos(void *arg) {
    leerYMostrarMatriz("Suma de matrices", "suma.txt");
    leerYMostrarMatriz("Resta de matrices", "resta.txt");
    leerYMostrarMatriz("Multiplicación de matrices", "multiplicacion.txt");
    leerYMostrarMatriz("Transpuesta de A", "transpuesta_A.txt");
    leerYMostrarMatriz("Transpuesta de B", "transpuesta_B.txt");
    leerYMostrarMatriz("Inversa de A", "inversa_A.txt");
    leerYMostrarMatriz("Inversa de B", "inversa_B.txt");
    return NULL;
}

int main() {
    srand(time(NULL));
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 101;
            B[i][j] = rand() % 101;
        }
    }
    printf("\nMatriz A:\n");
    imprimirMatriz(A);
    printf("\nMatriz B:\n");
    imprimirMatriz(B);
    pthread_t sumaHilo, restaHilo, multiplicacionHilo, transpuestaHilo, inversaHilo, lecturaHilo;
    pthread_create(&sumaHilo, NULL, sumarMatrices, NULL);
    pthread_create(&restaHilo, NULL, restarMatrices, NULL);
    pthread_create(&multiplicacionHilo, NULL, multiplicarMatrices, NULL);
    pthread_create(&transpuestaHilo, NULL, transponerMatriz, NULL);
    pthread_create(&inversaHilo, NULL, InversaMatriz, NULL);
    pthread_join(sumaHilo, NULL);
    pthread_join(restaHilo, NULL);
    pthread_join(multiplicacionHilo, NULL);
    pthread_join(transpuestaHilo, NULL);
    pthread_join(inversaHilo, NULL);
    pthread_create(&lecturaHilo, NULL, leerArchivos, NULL);
    pthread_join(lecturaHilo, NULL);
    return 0;
}

```

```
said@said-VirtualBox:~/Descargas$ ./mat
```

Matriz A:

14	65	21	13	76	65	52	71	15	61
53	59	56	52	28	43	64	94	75	41
71	22	65	13	0	77	20	96	33	59
51	70	43	33	1	66	12	27	12	24
81	98	2	8	38	29	36	71	95	28
43	67	47	100	66	37	90	79	59	26
55	52	20	1	25	8	22	57	34	94
11	79	97	92	11	17	39	89	58	5
85	85	55	7	29	22	63	39	91	80
15	66	99	80	9	6	65	32	98	42

Matriz B:

93	85	0	90	99	40	26	59	42	30
80	77	31	17	72	43	75	52	73	16
73	89	59	81	74	19	2	76	13	27
69	15	29	84	40	38	88	78	6	79
0	67	59	1	68	90	65	79	3	62
94	22	60	59	90	12	0	59	88	97
55	57	22	81	29	35	75	35	22	67
97	67	24	55	18	32	70	59	56	84
14	73	7	56	67	6	67	59	98	68
63	54	54	57	85	60	91	87	5	93

Suma de matrices:

107	150	21	103	175	105	78	130	57	91
133	136	87	69	100	86	139	146	148	57
144	111	124	94	74	96	22	172	46	86
120	85	72	117	41	104	100	105	18	103
81	165	61	9	106	119	101	150	98	90
137	89	107	159	156	49	90	138	147	123
110	109	42	82	54	43	97	92	56	161
108	146	121	147	29	49	109	148	114	89
99	158	62	63	96	28	130	98	189	148
78	120	153	137	94	66	156	119	103	135

Resta de matrices:

-79	-20	21	-77	-23	25	26	12	-27	31
-27	-18	25	35	-44	0	-11	42	2	25
-2	-67	6	-68	-74	58	18	20	20	32
-18	55	14	-51	-39	28	-76	-51	6	-55
81	31	-57	7	-30	-61	-29	-8	92	-34
-51	45	-13	41	-24	25	90	20	-29	-71
0	-5	-2	-80	-4	-27	-53	22	12	27
-86	12	73	37	-7	-15	-31	30	2	-79
71	12	48	-49	-38	16	-4	-20	-7	12
-48	12	45	23	-76	-54	-26	-55	93	-51

Multiplicación de matrices:

28842	26891	18262	21503	28134	19710	26791	28856	18527	30212
37638	35269	17276	34133	33551	18891	32447	35034	25668	35158
35834	28570	15675	29775	30407	13949	20570	29274	21632	29868
26922	20231	11975	20427	24277	10799	15409	21273	16892	19002
30758	33169	12043	25276	31804	17346	29217	28854	27514	27293
38245	35867	19557	35886	34572	23270	37926	38344	23678	38308
24693	24956	11942	20214	25061	15638	23555	24311	14629	22743
34275	30832	16179	30519	26957	15231	28712	31650	21122	28310
34833	38364	16393	32369	38263	20039	32028	34585	25443	31005
30683	32374	16250	32481	30512	15648	30666	32745	20806	29252

Transpuesta de A:

```
14 53 71 51 81 43 55 11 85 15
65 59 22 70 98 67 52 79 85 66
21 56 65 43 2 47 20 97 55 99
13 52 13 33 8 100 1 92 7 80
76 28 0 1 38 66 25 11 29 9
65 43 77 66 29 37 8 17 22 6
52 64 20 12 36 90 22 39 63 65
71 94 96 27 71 79 57 89 39 32
15 75 33 12 95 59 34 58 91 98
61 41 59 24 28 26 94 5 80 42
```

Transpuesta de B:

```
93 80 73 69 0 94 55 97 14 63
85 77 89 15 67 22 57 67 73 54
0 31 59 29 59 60 22 24 7 54
90 17 81 84 1 59 81 55 56 57
99 72 74 40 68 90 29 18 67 85
40 43 19 38 90 12 35 32 6 60
26 75 2 88 65 0 75 70 67 91
59 52 76 78 79 59 35 59 59 87
42 73 13 6 3 88 22 56 98 5
30 16 27 79 62 97 67 84 68 93
```

Inversa de A:

```
-32.09 -46.20 -3.22 -77.51 43.16 -67.90 55.76 74.25 -62.46 -38.70
-56.83 9.22 25.35 74.05 -13.54 19.75 46.29 -61.90 31.09 47.88
-59.71 -57.11 -1.50 -5.87 -78.24 33.14 -40.31 -63.20 84.31 -7.32
-18.83 -29.03 25.57 22.48 -42.07 83.17 5.16 -27.56 -30.91 29.94
-5.35 -83.85 -85.87 28.20 -49.39 -13.45 -8.33 -30.46 27.76 60.04
34.78 -50.59 61.58 60.74 81.84 42.69 -59.43 -45.54 41.51 -19.89
-58.13 43.78 69.03 83.53 -64.95 -24.48 51.71 -25.89 -62.26 -38.62
-62.69 -83.67 -83.06 18.46 -28.97 10.96 76.39 45.98 -59.94 -56.24
21.41 57.96 59.61 -56.53 64.17 72.41 -62.07 66.39 3.64 -15.79
5.68 -70.99 53.26 -85.93 -51.46 -30.32 17.68 -75.51 -77.83 28.28
```

Inversa de B:

```
-2.61 0.31 0.08 0.23 -1.82 2.94 1.92 -0.27 0.61 -2.06
-2.80 -2.62 2.42 2.19 1.52 3.05 0.37 -3.07 -0.79 -0.22
0.47 -3.12 -1.99 -1.38 -0.21 -0.05 0.60 2.85 -0.30 1.91
0.57 2.86 1.61 -2.30 0.98 1.74 -0.43 2.32 -2.44 -2.67
1.06 0.23 0.48 -0.51 -1.86 -0.84 -2.05 -2.02 0.20 -0.84
-0.17 -0.96 1.16 -2.55 -1.49 -1.07 -2.82 -1.30 -1.70 -2.63
0.28 -0.86 -1.39 1.74 1.04 -1.62 1.15 1.90 -1.67 2.05
-2.39 1.99 2.59 1.36 -2.13 -0.82 -2.42 -0.71 2.81 -1.58
2.44 -0.87 2.36 -1.61 2.03 1.95 -0.17 2.77 0.17 -2.95
1.85 -2.12 1.69 2.12 -0.80 -1.09 1.69 -0.52 -1.55 1.58
```


WINDOWS:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <windows.h>

#define N 10
int A[N][N];
int B[N][N];
int resultadoSuma[N][N];
int resultadoResta[N][N];
int resultadoMultiplicacion[N][N];
int resultadoT1[N][N];
int resultadoT2[N][N];
typedef struct {
    int matriz[N][N];
    double inversa[N][N];
    double determinante;
} Inversa;

void imprimirMatriz(int A[N][N]){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d\t", A[i][j]);
        }
        printf("\n");
    }
}

void escribirMatrizInt(const char *nombre_archivo, int matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%d ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}
```

```

void escribirMatrizDouble(const char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

void leerYMostrarMatriz(const char *titulo, const char *nombre_archivo) {
    FILE *archivo = fopen(nombre_archivo, "r");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        return;
    }
    printf("\n%s:\n", titulo);
    char linea[256];
    while (fgets(linea, sizeof(linea), archivo)) {
        printf("%s", linea);
    }
    fclose(archivo);
}

DWORD WINAPI sumarMatrices(LPVOID arg) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultadoSuma[i][j] = A[i][j] + B[i][j];
        }
    }
    escribirMatrizInt("suma.txt", resultadoSuma);
    return 0;
}

```

```

DWORD WINAPI restarMatrices(LPVOID arg) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultadoResta[i][j] = A[i][j] - B[i][j];
        }
    }
    escribirMatrizInt("resta.txt", resultadoResta);
    return 0;
}

DWORD WINAPI multiplicarMatrices(LPVOID arg) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultadoMultiplicacion[i][j] = 0;
            for (int k = 0; k < N; k++) {
                resultadoMultiplicacion[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    escribirMatrizInt("multiplicacion.txt", resultadoMultiplicacion);
    return 0;
}

DWORD WINAPI transponerMatriz(LPVOID arg) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultadoT1[j][i] = A[i][j];
            resultadoT2[j][i] = B[i][j];
        }
    }
    escribirMatrizInt("transpuesta_A.txt", resultadoT1);
    escribirMatrizInt("transpuesta_B.txt", resultadoT2);
    return 0;
}

```

```

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++) {
        for (int col = 0; col < n; col++) {
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1) {
                    j = 0;
                    i++;
                }
            }
        }
    }
}

int DeterminanteMatriz(int matriz[N][N], int n) {
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];

    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++) {
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = signo * DeterminanteMatriz(temp, n - 1);
        }
    }
}

```

```

DWORD WINAPI calcularInversa(LPVOID arg) {
    Inversa *datos = (Inversa *)arg;
    double determinante = (double)DeterminanteMatriz(datos->matriz, N);
    datos->determinante = determinante;
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return 0;
    }
    double adjunta[N][N];
    AdjuntaMatriz(datos->matriz, adjunta, N);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            datos->inversa[i][j] = adjunta[i][j] / determinante;
        }
    }
    return 0;
}

DWORD WINAPI InversaMatriz(LPVOID arg) {
    Inversa datosA = {.matriz = {0}, .inversa = {0}, .determinante = 0};
    Inversa datosB = {.matriz = {0}, .inversa = {0}, .determinante = 0};
    memcpy(datosA.matriz, A, sizeof(A));
    memcpy(datosB.matriz, B, sizeof(B));
    HANDLE thread1, thread2;
    thread1 = CreateThread(NULL, 0, calcularInversa, &datosA, 0, NULL);
    thread2 = CreateThread(NULL, 0, calcularInversa, &datosB, 0, NULL);
    WaitForSingleObject(thread1, INFINITE);
    WaitForSingleObject(thread2, INFINITE);

    if (datosA.determinante != 0) {
        escribirMatrizDouble("inversa_A.txt", datosA.inversa);
    }
    if (datosB.determinante != 0) {
        escribirMatrizDouble("inversa_B.txt", datosB.inversa);
    }
    return 0;
}

```

```

DWORD WINAPI leerArchivos(LPVOID arg) {
    leerYMostrarMatriz("Suma de matrices", "suma.txt");
    leerYMostrarMatriz("Resta de matrices", "resta.txt");
    leerYMostrarMatriz("Multiplicacion de matrices", "multiplicacion.txt");
    leerYMostrarMatriz("Transpuesta de A", "transpuesta_A.txt");
    leerYMostrarMatriz("Transpuesta de B", "transpuesta_B.txt");
    leerYMostrarMatriz("Inversa de A", "inversa_A.txt");
    leerYMostrarMatriz("Inversa de B", "inversa_B.txt");
    return 0;
}

int main() {
    srand(time(NULL));
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 101;
            B[i][j] = rand() % 101;
        }
    }
    printf("\nMatriz A:\n");
    imprimirMatriz(A);
    printf("\nMatriz B:\n");
    imprimirMatriz(B);
    HANDLE h1, h2, h3, h4, h5;
    h1 = CreateThread(NULL, 0, sumarMatrices, NULL, 0, NULL);
    h2 = CreateThread(NULL, 0, restarMatrices, NULL, 0, NULL);
    h3 = CreateThread(NULL, 0, multiplicarMatrices, NULL, 0, NULL);
    h4 = CreateThread(NULL, 0, transponerMatriz, NULL, 0, NULL);
    h5 = CreateThread(NULL, 0, InversaMatriz, NULL, 0, NULL);
    WaitForSingleObject(h1, INFINITE);
    WaitForSingleObject(h2, INFINITE);
    WaitForSingleObject(h3, INFINITE);
    WaitForSingleObject(h4, INFINITE);
    WaitForSingleObject(h5, INFINITE);
    HANDLE h6 = CreateThread(NULL, 0, leerArchivos, NULL, 0, NULL);
    WaitForSingleObject(h6, INFINITE);
    return 0;
}

```

```
C:\Users\vbcs1>mat
```

```
Matriz A:
```

90	57	92	81	0	73	53	30	2	61
100	89	11	94	64	8	50	36	13	81
13	100	89	3	85	70	60	52	84	52
8	5	24	85	16	67	96	59	88	45
68	100	10	13	21	65	28	58	5	67
15	3	40	41	40	10	51	91	25	25
14	6	9	85	46	73	3	46	97	61
13	81	8	36	29	58	43	32	16	95
26	71	21	63	43	14	82	61	57	83
68	99	57	66	17	56	26	57	33	3

```
Matriz B:
```

86	32	71	15	68	75	86	98	34	24
100	36	98	26	72	14	44	25	5	60
92	16	41	46	41	12	54	66	74	34
94	35	29	14	95	86	11	81	45	59
34	7	74	3	53	47	62	35	68	74
22	23	29	10	53	14	89	27	89	14
38	96	88	39	12	93	6	63	78	70
45	68	9	27	50	1	89	70	77	62
6	26	46	89	19	40	83	37	34	22
61	26	54	95	0	52	49	100	78	72

```
Suma de matrices:
```

176	89	163	96	68	148	139	128	36	85
200	125	109	120	136	22	94	61	18	141
105	116	130	49	126	82	114	118	158	86
102	40	53	99	111	153	107	140	133	104
102	107	84	16	74	112	90	93	73	141
37	26	69	51	93	24	140	118	114	39
52	102	97	124	58	166	9	109	175	131
58	149	17	63	79	59	132	102	93	157
32	97	67	152	62	54	165	98	91	105
129	125	111	161	17	108	75	157	111	75

```
Resta de matrices:
```

4	25	21	66	-68	-2	-33	-68	-32	37
0	53	-87	68	-8	-6	6	11	8	21
-79	84	48	-43	44	58	6	-14	10	18
-86	-30	-5	71	-79	-19	85	-22	43	-14
34	93	-64	10	-32	18	-34	23	-63	-7
-7	-20	11	31	-13	-4	-38	64	-64	11
-24	-90	-79	46	34	-20	-3	-17	19	-9
-32	13	-1	9	-21	57	-46	-38	-61	33
20	45	-25	-26	24	-26	-1	24	23	61
7	73	3	-29	17	4	-23	-43	-45	-69

Multiplicacion de matrices:

```
38221 19684 28534 17778 27734 24851 28747 36462 31565 24515
38239 20194 33663 17682 29052 29500 27376 37072 27385 30358
32314 20582 35199 24046 25149 20372 36365 30484 35269 29801
22980 22134 23091 20706 20136 24805 25430 29681 31482 23999
27925 16537 25688 14443 21358 15981 27046 26076 23396 21693
18412 15557 15245 11983 15058 14327 19108 22184 22725 18981
20279 13308 18371 18625 20314 19113 27390 25680 26454 20031
24565 15447 23999 16830 17495 17087 21559 24365 23965 22712
30226 22212 29949 22156 21864 25114 25818 32540 29303 29445
32940 17677 26144 13526 28017 18380 28161 27503 23908 21742
```

Transpuesta de A:

```
90 100 13 8 68 15 14 13 26 68
57 89 100 5 100 3 6 81 71 99
92 11 89 24 10 40 9 8 21 57
81 94 3 85 13 41 85 36 63 66
0 64 85 16 21 40 46 29 43 17
73 8 70 67 65 10 73 58 14 56
53 50 60 96 28 51 3 43 82 26
30 36 52 59 58 91 46 32 61 57
2 13 84 88 5 25 97 16 57 33
61 81 52 45 67 25 61 95 83 3
```

Transpuesta de B:

```
86 100 92 94 34 22 38 45 6 61
32 36 16 35 7 23 96 68 26 26
71 98 41 29 74 29 88 9 46 54
15 26 46 14 3 10 39 27 89 95
68 72 41 95 53 53 12 50 19 0
75 14 12 86 47 14 93 1 40 52
86 44 54 11 62 89 6 89 83 49
98 25 66 81 35 27 63 70 37 100
34 5 74 45 68 89 78 77 34 78
24 60 34 59 74 14 70 62 22 72
```

Inversa de A:

```
-1.66 1.67 0.22 0.07 0.78 -0.71 1.30 -1.18 -0.96 -0.70
-0.87 -1.66 -0.10 1.44 1.23 1.37 -1.48 -0.71 -0.93 -1.17
-0.18 0.18 -0.03 -0.94 1.32 1.01 -1.58 0.99 1.41 0.64
1.53 1.07 1.16 0.99 0.34 -0.98 1.58 -0.80 1.34 -1.68
0.39 -1.29 -1.25 -0.09 -0.59 -0.83 0.36 0.53 0.39 0.19
-1.27 0.66 0.76 1.35 -1.56 0.99 1.64 0.48 0.98 -1.62
0.98 0.97 0.99 0.07 -0.59 -1.71 -0.94 -0.08 0.44 1.56
1.18 0.11 -0.17 0.81 -0.73 0.09 1.53 1.28 1.04 -0.85
0.01 0.03 -1.11 1.67 0.77 0.39 -1.57 1.69 -0.01 -0.12
-1.13 1.01 -0.30 0.04 0.52 -0.61 1.14 -0.76 -1.40 -0.79
```



```

Inversa de B:
1.24 -0.58 1.60 -0.46 0.75 0.74 1.30 -0.73 0.78 1.09
0.47 -0.96 -0.77 -0.49 -0.27 1.35 -1.13 -0.85 -1.56 -0.07
-1.61 -0.51 0.09 0.70 -0.19 -0.07 -0.99 -1.49 1.05 -0.21
0.28 0.96 -0.94 -1.69 -0.14 -0.01 -0.36 -0.14 -0.53 -1.13
-0.62 -1.03 -1.42 0.16 1.12 1.22 -0.62 0.42 0.66 -1.69
1.25 -1.40 1.46 -0.61 1.60 0.01 0.44 -0.30 0.19 0.83
1.18 1.36 1.30 1.67 0.19 -0.56 -1.24 -1.11 1.08 0.98
0.68 1.45 -1.27 0.78 -0.95 1.45 1.35 -0.34 0.38 0.47
0.68 1.31 1.12 0.46 -0.13 0.09 -0.52 -0.20 -0.74 -1.35
0.54 0.07 -0.30 -0.81 1.26 1.19 -1.54 1.01 1.50 -0.92

```

TIEMPO HILOS:

TIEMPO PROCESOS:

LINUX:

```

real    0m2,109s
user    0m2,146s
sys     0m1,496s

```

```

real    0m2,661s
user    0m2,646s
sys     0m0,016s

```

WINDOWS:

```

Seconds      : 2
Milliseconds : 174
Ticks        : 21742648

```

```

Seconds      : 4
Milliseconds : 201
Ticks        : 42014814

```

En Linux, los hilos son más rápidos que los procesos debido a que comparten memoria y recursos, lo que reduce el tiempo de ejecución en comparación con los procesos que requieren su propio espacio de memoria. Aunque en Windows los hilos también resultan más rápidos que los procesos, la diferencia en el tiempo de ejecución es menos pronunciada debido a la sobrecarga adicional que implica la gestión de hilos en este sistema operativo. Esto sugiere que, aunque los hilos son generalmente más eficientes que los procesos en términos de uso de recursos y tiempos de ejecución, el sistema operativo juega un papel importante en la optimización de estos procesos. En conclusión, los hilos son la mejor opción cuando se busca eficiencia en el tiempo de ejecución, especialmente en Linux, donde la gestión de hilos está mejor optimizada, mientras que en Windows la diferencia no es tan

significativa, aunque sigue siendo más favorable usar hilos que procesos en tareas concurrentes.

Conclusión

A lo largo de esta práctica hemos explorado los conceptos y diferencias entre el uso de procesos y hilos para realizar tareas concurrentes en Linux y Windows, específicamente en el contexto de operaciones matemáticas sobre matrices. A través de los programas implementados, hemos observado cómo la ejecución de operaciones como la suma, resta, multiplicación, transposición, y cálculo de la inversa de matrices se ve influenciada por el tipo de mecanismo de concurrencia utilizado.

En Linux, los hilos demostraron ser significativamente más rápidos que los procesos debido a la eficiencia con la que el sistema operativo maneja la memoria compartida entre hilos dentro de un mismo proceso. Los hilos permiten un uso más eficiente de los recursos del sistema, ya que comparten el mismo espacio de memoria, lo que reduce la sobrecarga de crear y gestionar procesos independientes. Por otro lado, los procesos en Linux requieren un espacio de memoria separado, lo que aumenta el tiempo de ejecución debido a la mayor sobrecarga de gestión de recursos.

En Windows, aunque los hilos también fueron más rápidos que los procesos, la diferencia no fue tan significativa como en Linux. Esto puede deberse a la diferencia en la manera en que ambos sistemas operativos manejan la concurrencia. En Windows, los procesos y hilos tienen más restricciones en cuanto a la optimización del uso de recursos, lo que puede generar una mayor sobrecarga al usar hilos en comparación con Linux.

La comparación de los tiempos de ejecución reveló que, en general, los hilos son la opción más eficiente cuando se trabaja con tareas concurrentes, ya que permiten una mayor reutilización de los recursos del sistema. Sin embargo, la eficiencia de los hilos frente a los procesos también depende del sistema operativo y de cómo gestiona la concurrencia.

En conclusión, los hilos son preferibles en la mayoría de los casos cuando se busca una ejecución más rápida y eficiente de tareas concurrentes, especialmente en entornos como Linux. Los procesos, aunque más independientes y aislados, tienden a ser más lentos debido a la mayor sobrecarga de gestión de recursos. La práctica nos muestra la importancia de elegir el mecanismo de concurrencia adecuado según las necesidades del sistema y las características de las tareas a realizar.